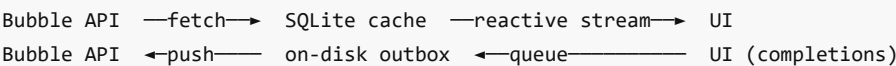


BlockPro — Data Loading & Storage Report

How the Flutter app **loads** data from the Bubble backend, **stores** it on the device, and the **use cases** that trigger each of these.

Core principle: offline-first / cache-first

The local **SQLite database is the single source of truth that the UI reads from**. The API is only ever used to *fill or refresh that cache* in the background — screens never wait on the network to render. Writes (completed inspections) go the other way: they are saved locally first and pushed to the server when connectivity allows.



1. Where the data is stored

All requests go through [ApiRepository](#) (prefixes `ApiConfig.baseUrl` from `BUBBLE_API_BASE_URL` in `.env`, attaches the `Bearer` token). Responses are parsed by [api_parsers.dart](#) and written to one of these stores:

Terminology: *block* (UI/API wording) and *building* (DB table name) are the same entity; this doc uses both. Likewise "cache", "SQLite", and "the DB" all mean the local database.

SQLite (Drift) — `<appDocs>/blockpro.sqlite`

The server-data cache. Defined in [database.dart](#).

Table	Holds	Source
<code>buildings</code>	block list	API (endpoint 2)
<code>assets</code>	inspectable assets per block	API (endpoint 3) + local completion write-back
<code>chapters</code>	checklist sections per asset	API (endpoint 4)
<code>questions</code>	checklist questions (+ existingRemedials as a JSON-text blob)	API (endpoint 4)
<code>draft_inspections</code>	an in-progress inspection (one per asset)	local only — never synced
<code>draft_answers</code>	answers/photo-paths for a draft	local only — never synced

- Every cached row carries a `lastSyncedAt` timestamp.
- On `assets`, `assetRegisterItems` and `tooltipUrls` are stored as **opaque JSON-text blobs** (not exploded into columns).
- The DB is a **disposable cache**: any schema upgrade *drops and recreates* every table (current `schemaVersion = 8`), and [clearAllData](#) wipes **the DB** on logout and on manual refresh (the user-triggered full reload, \$5). Lost data is just re-fetched.

Outside the database (survives `clearAllData` and schema wipes)

Store	Location	Holds
-------	----------	-------

Auth token / user	SharedPreferences	token, expiry, uid, user data
Theme preference	SharedPreferences	light/dark/system choice (brightness_mode)
Offline outbox	<appDocs>/outbox/outbox.json	queued completed inspections — durable, wipe-proof JSON manifest (OutboxStore)
Completion photos	<appDocs>/outbox/<submissionId>/	photos for a <i>queued</i> completion, keyed by submissionId so a later draft can't overwrite them (CompletionPhotoStore)
Draft photos	<appDocs>/draft_photos/<assetId>/	photos for an <i>in-progress</i> inspection, keyed by assetId

On launch, an **expired or missing token** forces sign-out and re-login. Sign-out is the **one** event that runs a *full* purge — it explicitly clears the outbox and photo stores *as well as* wiping the DB — so these otherwise wipe-proof stores (untouched by manual refresh or schema migrations) can't leak to the next user. There is no token-refresh flow (`_refreshToken` is persisted but never populated or used).

2. API endpoints

#	Endpoint	Method	Caller	Writes to
1	app_login	POST	AuthRepository.login	auth token → SharedPreferences
2	app_fetchbuildings	GET	syncBuildings	buildings
3	app_fetch_all_assets?block_id=<id>	GET	syncAssetsForBuilding	assets
4	app_fetch_checklist_single?asset_id=<id>	GET	syncChecklistForAsset	chapters, questions
5	app_upload-image_Adam	POST	replayCompletion	none (returns an image_id)
6	app_completed-inspection	POST	replayCompletion	assets (server due/yellow dates)

Endpoints **2–4** are the read path that *populates the cache*. Endpoints **5–6** are the write path (submitting a completed inspection).

3. The full-sync pipeline

[SyncRepository.syncAll](#) runs three phases in order — this is the unit the "big" triggers invoke:

Phase 1	app_fetchbuildings	→ buildings	(1 call)
Phase 2	app_fetch_all_assets per block	→ assets	(1 call / building, pool of 5)
Phase 3	app_fetch_checklist_single	→ chapters, questions	(1 call / STALE asset, pool of 5)

Each fetch → parse → **upsert** (insert-or-replace), so re-running is idempotent. Checklists are the exception: each asset's chapters and questions are **deleted and re-inserted** (not upserted), so removed or renamed items don't linger.

4. Loading strategy & data freshness

How hard each call works to *avoid* redundant downloads:

- **Buildings (endpoint 2):** always a full re-fetch of every building. No incremental diff.
- **Assets (endpoint 3):** always a full re-fetch of *all* assets for the block. `assetLastModified` is stored but **not** used to skip the call. (Don't confuse it with `checklistLastModified` — a *different* field on the same asset that *does* drive incremental checklist sync, below.)
- **Checklists (endpoint 4): incremental.** Phase 2 records each asset's fresh `checklistLastModified` ; Phase 3 only fetches checklists whose timestamp changed or was never synced (`needsResync`). The per-screen trigger adds a second guard (`isChecklistCacheCurrent`) that skips the network entirely when the cached checklist is already current.
- **Manual refresh** wipes the DB *first*, so the incremental diff has nothing to compare against and **every** checklist re-downloads — a deliberate full reload.
- `forceFullChecklists: true` bypasses the incremental check (debug audit only).

On failure: a fetch that errors is caught and logged, not surfaced — the sync silently no-ops for that item and the UI keeps showing the last cached rows. The offline/online signal that gates outbox draining is itself inferred from whether these calls succeed or fail (`api_repository.dart`).

5. Use cases — when each call fires

Each endpoint fires either as part of the composite `syncAll` pipeline or from its own per-screen trigger. The per-endpoint subsections below add only the *standalone* triggers — they point back to `syncAll` but don't re-describe its cases.

Endpoint 1 — `app_login`

- **User taps "Sign in".** Authenticates and stores the token; no DB rows written.

`syncAll` (endpoints 2 → 3 → 4 together)

Use case	Trigger	Notes
First login / initial download	<code>initialSyncNotifierProvider.runSync</code> on the initial-sync screen	Shown when the DB has no buildings (<code>needsInitialSyncProvider</code>). All assets are new → all checklists fetched.
Manual full refresh	<code>refreshNotifierProvider.run</code> from the refresh dialog on the Blocks list	Wipes the whole DB first , then re-downloads everything; re-asserts queued completions and drains the outbox afterward. Cancellable.
Debug data audit	<code>DebugAuditButtonState.run</code> on the About screen	<code>syncAll(forceFullChecklists: true)</code> — re-fetches <i>every</i> checklist and dumps an audit report.

Endpoint 2 — `app_fetchbuildings` (`syncBuildings`)

- As **Phase 1** of `syncAll` (all three use cases above).

- **Blocks list, first launch only** — `PaginatedBuildingsNotifier.init` syncs only when the DB is empty (avoids a redundant call when the sync screen already populated it).
- **Pull-to-refresh** on the Blocks list — `PaginatedBuildingsNotifier.refresh` .

Endpoint 3 — `app_fetch_all_assets` (`syncAssetsForBuilding`)

- As **Phase 2 of** `syncAll` , pooled 5-at-a-time across all buildings.
- **Opening a block's asset list** — `PaginatedAssetsNotifier.init` fires a background sync for that one block **every time the screen mounts**.
- **Pull-to-refresh** on the asset list — `PaginatedAssetsNotifier.refresh` .

Endpoint 4 — `app_fetch_checklist_single` (`syncChecklistForAsset`)

- As **Phase 3 of** `syncAll` , pooled 5-at-a-time across stale assets.
- **Opening an asset's checklist** — `checklistChaptersStreamProvider` and `checklistCountProvider` each trigger a sync on first watch, skipping the call when the cached checklist is current.

Endpoints 5 & 6 — photo upload + completed inspection (`replayCompletion`)

A completed inspection is **queued locally** and sent through the **outbox drainer** — the single send path. The flow:

1. On **"Complete"**, the inspection is enqueued in the outbox, the `assets` row is updated **optimistically** (`markCompleted`), and the draft is deleted.
2. The drainer uploads each photo (endpoint 5), then POSTs the inspection (endpoint 6). On success it overwrites the `assets` row with the server's authoritative due/yellow dates and removes the outbox entry.

`OutboxDrainer.drain` is triggered by:

Use case	Trigger
App launch	<code>main</code> (flushes anything queued offline last session)
App returns to foreground	<code>didChangeAppLifecycleState</code>
Network regained (offline → online)	<code>outboxDrainTriggerProvider</code>
Inspector taps "Complete"	<code>inspection_provider</code> — sends immediately when online, stays queued when offline
After a manual refresh	<code>refresh_sync_provider</code>
Manual retry / re-send of a failed or needs-review entry	<code>block inspections screen</code>

6. How the UI reads the data back

Screens never read the API directly — they watch **reactive Drift streams** over the SQLite cache, so they render instantly (even offline) and auto-update the moment a background sync writes new rows:

- Lists are **paginated locally** (page size 20) via `watchBuildingsPaginated` / `watchAssetsPaginated` .
- Search runs against the cache (`watchBuildingsMatching` / `watchAssetsMatching`).
- The checklist screen combines the `chapters` + `questions` streams into a hierarchical view (`checklist_provider.dart`).
- A scanned QR code is resolved to an asset purely from the local DB (`scannedAssetProvider`) — no network round-trip.

Debug note: in debug builds every fetch is mirrored to a `data_audit/` folder via [data_audit.dart](#) (raw response, parser coverage, final DB state) — instrumentation only, no effect on sync.